

Project: Manual Deployment Using Docker

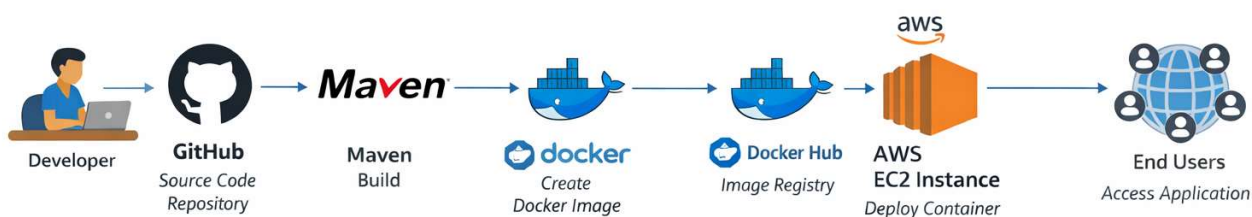
Definition:

This project demonstrates the deployment of a web application using containerization technology in a cloud environment. The source code of the Online Bookstore application is stored in GitHub and is manually cloned to an EC2 instance hosted on Amazon Web Services. The application is built using Maven and then containerized using Docker to ensure consistent execution across different environments. The Docker image can also be stored and shared through Docker Hub. Finally, the container is deployed and executed on the EC2 instance, allowing users to access the application through a web browser. This approach demonstrates how containerization simplifies application deployment and ensures portability across environments.

Scope of the Project:

- To demonstrate application deployment in a cloud environment using Amazon Web Services.
- To manage and store the application source code using GitHub.
- To build the web application using the Maven build tool.
- To containerize the application using Docker for consistent execution across environments.
- To store and manage Docker images using Docker Hub.
- To deploy the containerized application on an EC2 instance.
- To reduce dependency issues by running the application inside a Docker container.
- To provide a simple and efficient method for deploying web applications in cloud environments.
- To demonstrate the basic workflow of modern DevOps deployment practices.
- To improve portability and scalability of the application deployment process.

Architecture Diagram:

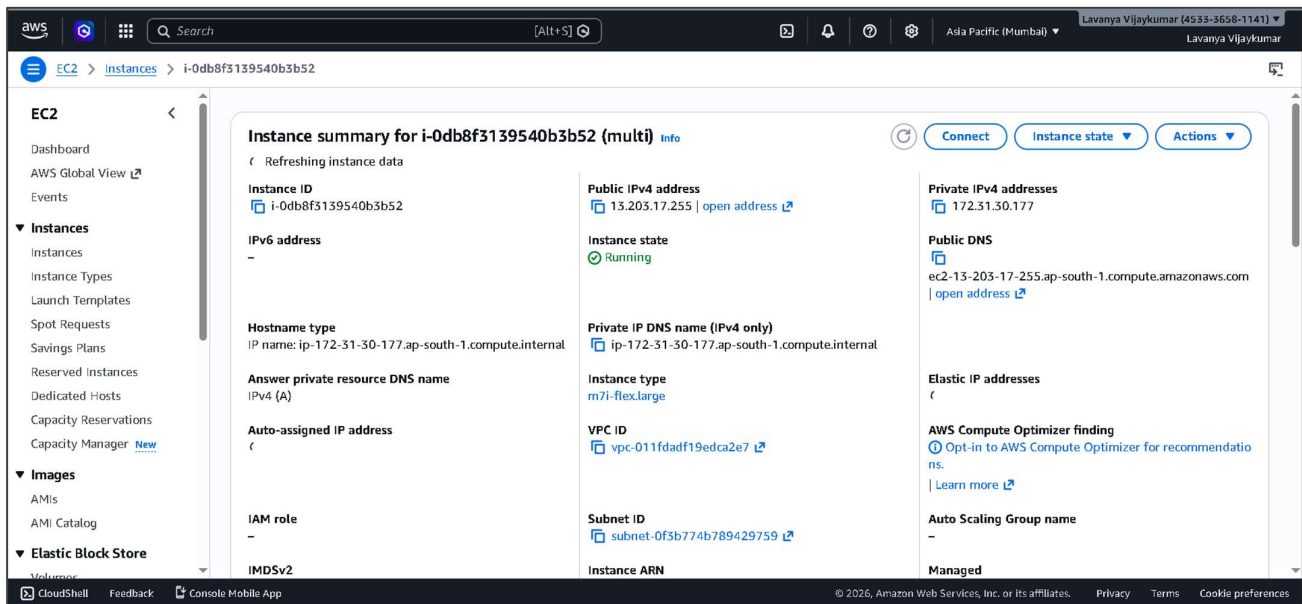


CI/CD Pipeline Deployment with Docker on AWS

Project Implementation steps:

STEP 1: Create AWS EC2 Instance

- Launch EC2 instance (Amazon Linux).
- Open ports in Security Group:
- 22 (SSH)
- 8080
- 8081
- Connect to instance



STEP 2: Install Required Software like git, maven, docker etc

```
[ec2-user@ip-172-31-30-177 ~]$ git --version
git version 2.50.1
[ec2-user@ip-172-31-30-177 ~]$ mvn --version
Apache Maven 3.8.4 (Red Hat 3.8.4-3.amzn2023.0.5)
Maven home: /usr/share/maven
Java version: 17.0.18, vendor: Amazon.com Inc., runtime: /usr/lib/jvm/java-17-amazon-corretto.x86_64
Default locale: en, platform encoding: UTF-8
OS name: "linux", version: "6.1.163-186.299.amzn2023.x86_64", arch: "amd64", family: "unix"
[ec2-user@ip-172-31-30-177 ~]$ docker --version
Docker version 25.0.14, build 0bab007
[ec2-user@ip-172-31-30-177 ~]$
```

STEP 3: Clone Project

- Clone repository from GitHub:
- `git clone https://github.com/Lavanya-Vijaykumar/onlinebookstore.git`
`cd onlinebookstore`

```

aws [Search] [Alt+S] Ask Amazon Q Asia Pacific (Mumbai) Lavanya Vijaykumar (4533-3658-1141) Lavanya Vijaykumar
[WARNING] bootstrap class path not set in conjunction with -source 8
[INFO] --- maven-resources-plugin:2.6:testResources (default-testResources) @ onlinebookstore ---
[WARNING] Using platform encoding (UTF-8 actually) to copy filtered resources, i.e. build is platform dependent!
[INFO] skip non existing resourceDirectory /home/ec2-user/onlinebookstore/src/test/resources
[INFO] --- maven-compiler-plugin:3.11.0:testCompile (default-testCompile) @ onlinebookstore ---
[INFO] No sources to compile
[INFO] --- maven-surefire-plugin:2.12.4:test (default-test) @ onlinebookstore ---
[INFO] No tests to run.
[INFO] --- maven-war-plugin:3.2.1:war (default-war) @ onlinebookstore ---
[INFO] Packaging webapp
[INFO] Assembling webapp [onlinebookstore] in [/home/ec2-user/onlinebookstore/target/onlinebookstore]
[INFO] Processing war project
[INFO] Copying webapp resources [/home/ec2-user/onlinebookstore/WebContent]
[INFO] Webapp assembled in [36 msec]
[INFO] Building war: /home/ec2-user/onlinebookstore/target/onlinebookstore.war
[INFO] --- maven-dependency-plugin:2.3:copy (default) @ onlinebookstore ---
[WARNING] The artifact xml-apis:xml-apis:jar:2.0.2 has been relocated to xml-apis:xml-apis:jar:1.0.b2
[INFO] Configured Artifact: com.github.jsimone:webapp-runner:8.0.30.2:jar
[INFO] Copying webapp-runner-8.0.30.2.jar to /home/ec2-user/onlinebookstore/target/dependency/webapp-runner.jar
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 2.447 s
[INFO] Finished at: 2026-03-08T17:53:14Z
[INFO] -----
[ec2-user@ip-172-31-30-177 onlinebookstore]$ ls target
classes dependency generated-sources maven-archiver maven-status onlinebookstore onlinebookstore.war

```

STEP 4: Create Dockerfile

```

aws [Search] [Alt+S] Ask Amazon Q Asia Pacific (Mumbai) Lavanya Vijaykumar (4533-3658-1141) Lavanya Vijaykumar
GNU nano 8.3 Dockerfile
FROM tomcat:9-jdk17

COPY target/onlinebookstore.war /usr/local/tomcat/webapps/

EXPOSE 8080

```

STEP 5: Build Docker Image

- docker build -t onlinebookstore .
- docker images
- docker run -d -p 8081:8080 onlinebookstoreCon

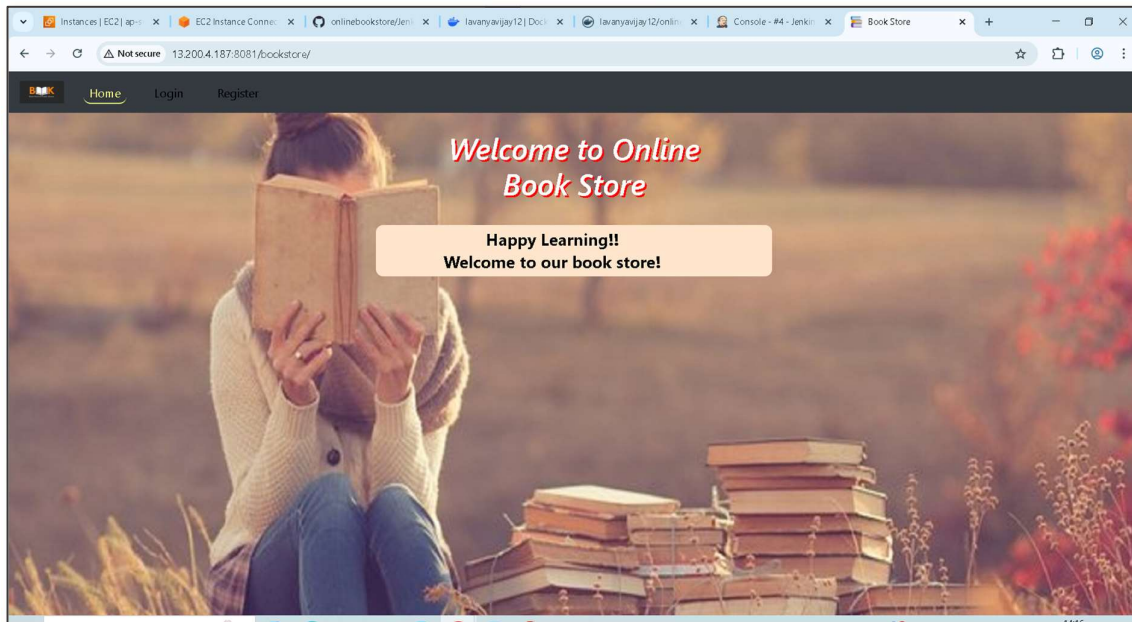
```

[ec2-user@ip-172-31-30-177 onlinebookstore]$ docker run -d -p 8081:8080 onlinebookstore
434ab78400377ab3706e8e46fcc4b5dfa74552a432717302631ff9c75db587fc
[ec2-user@ip-172-31-30-177 onlinebookstore]$ docker ps
CONTAINER ID   IMAGE             COMMAND                  CREATED        STATUS        PORTS                               NAMES
434ab7840037   onlinebookstore   "catalina.sh run"       8 seconds ago Up 7 seconds   0.0.0.0:8081->8080/tcp, :::8081->8080/tcp   amazing_colden

```

STEP 6: Access Application

- Open browser:
 - <http://EC2-PUBLIC-IP:8081/onlinebookstore>
- Application will run.



Conclusion:

This project demonstrates how a web application can be manually deployed using containerization technology in a cloud environment. The source code was managed through GitHub and the application was built using Maven. The application was then containerized using Docker, which ensured consistent execution across different environments. The Docker image was deployed on an EC2 instance provided by Amazon Web Services, allowing the application to be accessed through a web browser.

Through this project, the manual deployment process helped in understanding the workflow of application building, containerization, and cloud deployment. It also demonstrated how Docker simplifies dependency management and improves portability. Overall, the project provides a practical understanding of deploying containerized applications in a cloud infrastructure environment.